Master Data Science & Master Mathématiques – ENS Martil

Module : Apprentissage Automatique Probabiliste

MMedAgent-Lite

Système Multi-Agent RL pour le Diagnostic Médical

Radiologie & Anatomopathologie
2 Spécialités

Partie Mathématiques (8 personnes) : Fondements RL · C-MARL · FL · Distillation

Partie Data Science (10 personnes) : Prototype · RAG MedCPT · Distillation 7B→3B · FL Léger

Papier de base : MMedAgent-RL – Xia et al., ICLR 2026 (arXiv:2506.00555)
Modèle embarqué : Qwen2.5-VL-3B INT4 GGUF (2.1 Go) – CPU only
Datasets : VQA-RAD (radiologie) + PathVQA (pathologie)
RAG sélectionné : MedCPT-Query-Encoder (public, HF) + BM25 hybride
Encadrant : Pr. Abdellatif EL AFIA
Année : 2026

Contents

1 Contexte Général et Structure du Projet **5**
1.1 Problématique et Objectif 5
1.2 Organisation du Projet : Deux Profils Complémentaires 6
1.3 Les Deux Spécialités et leurs Datasets 7

2 Partie Mathématiques (8 Personnes) – Fondements Théoriques **8**
2.1 M1 – Fondements MDP et GRPO : Cadre Théorique 8
2.2 M2 – C-MARL : Convergence et Optimalité 9
2.3 M3 – Reward Shaping Online : Garanties 9
2.4 M4 – Federated Learning : Convergence FedAvg et FedProx 10
2.5 M5 – Théorie de la Distillation : RAG Médical et Modèle 11
2.6 M6 – Traitement du Bruit dans les Données Médicales 12
2.7 M7 – Sécurité et Confidentialité : DP Formel 12
2.8 M8 – Analyse Statistique et Métriques d'Évaluation 13

3 RAG Médical : État de l'Art et Justification du Choix **14**
3.1 Comparaison des Systèmes RAG Médicaux 2024–2025 14
3.2 Justification du Choix : MedRAG-Lite avec MedCPT-Query-Encoder . . 15
3.2.1 Pourquoi MedCPT comme retriever ? 15
3.2.2 Pourquoi ajouter BM25 en mode hybride ? 16
3.2.3 Pourquoi le Retriever MedCPT est-il adapté ? 16
3.3 Architecture MedRAG-Lite : Description Technique 16

4 Partie Data Science (10 Personnes) – Prototype Fonctionnel **18**
4.1 Groupe DS 1 – Données et Prétraitement (D1, D2) 18
4.2 Groupe DS 2 – Agents GP1, S1, S2 (D3, D4, D5) 19
4.3 Groupe DS 3 – GP2, C-MARL et Reward Online (D6, D7) 19
4.4 Groupe DS 4 – RAG Médical, Distillation Pratique et FL (D8, D9) . . . 20
4.5 Groupe DS 5 – Intégration, Évaluation et Rapport (D10) 22

5 Tableaux Résumés et Calendrier **23**
5.1 Tableau de Répartition Synthétique des 18 Personnes 23
5.2 Calendrier Unifié – 7 Semaines 24
5.3 Performances Attendues 25
5.4 Grille d'Évaluation (100 points) 26
5.5 Structure du Dépôt GitHub 26

6 Recommandations Stratégiques et Analyse Critique **28**

6.1 Recommandation 1 – Stratégie d'Intégration : Mitigation du Risque . . . 28
6.1.1 Approche "Feature Flags" – Pipeline Modulaire (D10) 28
6.1.2 Valider d'abord le "Happy Path" 29
6.2 Recommandation 2 – Optimisation Matérielle : Le Goulot CPU / 8 Go . 29
6.2.1 Réduire la Charge du GRPO : $G = 2$ en Prototypage 29
6.2.2 Gestion Agressive du KV-Cache 29
6.3 Recommandation 3 – Stabilité Algorithmique du RL 30
6.3.1 Geler le Reward Online au Démarrage 30
6.3.2 Simplifier la Transition de Curriculum si Nécessaire 30
6.4 Recommandation 4 – Fiabilité Médicale et Déploiement 31
6.4.1 Pénaliser Sévèrement les Hallucinations (RAG) 31
6.4.2 Structurer les Sorties : Format JSON pour l'Interopérabilité SIH . 31
6.4.3 Résumé des Actions par Priorité 32

7 Conclusion **33**
7.1 Synthèse 33
7.2 Justification des Choix Techniques 34

List of Figures

1.1 Architecture MMedAgent-Lite : GP1 → S1/S2 → GP2 avec RAG Médical, FL Léger et Reward Online Adaptatif 6

List of Tables

1.1 Deux profils, deux missions – répartition des 18 personnes 6
1.2 Spécialités et datasets de MMedAgent-Lite 7

3.1 Comparaison des principaux RAG médicaux – état de l'art 2024–2025 . . 15

5.1 Répartition synthétique – 8 Math + 10 DS 23
5.2 Calendrier détaillé – Math + Data Science, 7 semaines 24
5.3 Tableau de résultats attendus – à remplir lors du projet 25
5.4 Grille d'évaluation combinée Math + Data Science 26

6.1 Stratégie adaptative du nombre de rollouts GRPO 29
6.2 Stratégie de repli pour le curriculum C-MARL 31
6.3 Plan d'action stratégique – par priorité et responsable 32

7.1 Justification des choix techniques de MMedAgent-Lite 34

Chapter 1

Contexte Général et Structure du Projet

1.1 Problématique et Objectif

Le diagnostic médical multimodal est fondamentalement un problème multidisciplinaire : un radiologue lit les images, un pathologiste analyse les biopsies, et un médecin senior synthétise l'ensemble. Les modèles uniques de vision-langage (Med-LVLMs) échouent à reproduire cette collaboration : Qwen2.5-VL-7B n'atteint que 61.8% sur VQA-RAD et 60.5% sur PathVQA [1], bien en deça des spécialistes humains.

MMedAgent-RL [1] (ICLR 2026) propose la première architecture multi-agent basée sur RL pour ce problème : un médecin généraliste **GP1** oriente le cas vers les spécialistes, ceux-ci consultent indépendamment, et **GP2** synthétise pour prendre la décision finale en apprenant, via C-MARL, quand faire confiance au consensus et quand le corriger. Résultat publié : +23.6% vs SFT, 73.3% d'accuracy moyenne sur 5 benchmarks.

MMedAgent-Lite est notre version applicable en quelques semaines, avec 2 spécialités, un RAG médical de pointe, un Federated Learning léger et des stratégies de reward online.

1.2 Organisation du Projet : Deux Profils Complémentaires

Table 1.1: Deux profils, deux missions – répartition des 18 personnes

#	Profil	Mission	Livrables attendus
8	**Master Math.**	Formaliser et démontrer les fondements théoriques : convergence GRPO, optimalité C-MARL, garanties FL, stabilité LoRA, traitement bruit.	Notes de preuves, propositions/théorèmes, rapport math., code de simulation.
10	**Master DS**	Construire un prototype fonctionnel de bout en bout : prétraitement, agents, RAG médical, reward online, FL léger, évaluation.	Scripts Python, checkpoints modèles, dashboards, rapport technique.

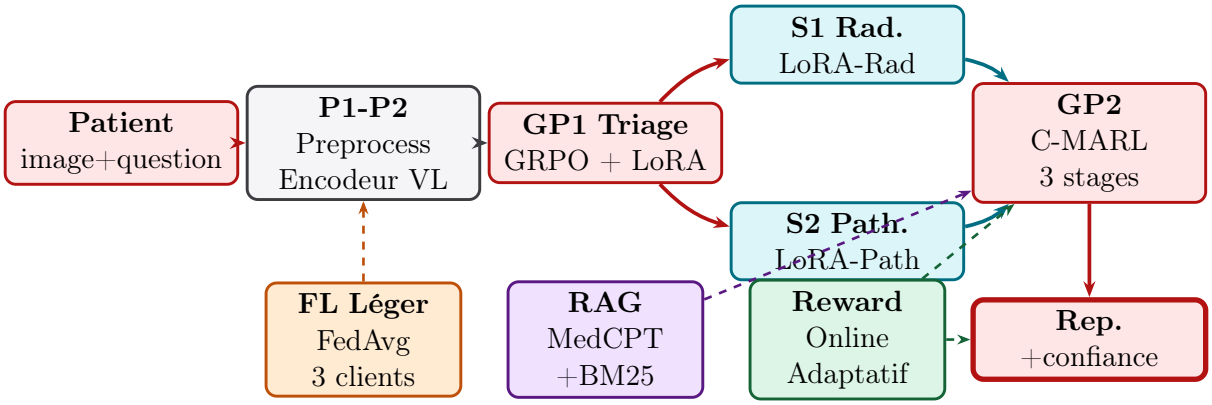


Figure 1.1: Architecture MMedAgent-Lite : GP1 → S1/S2 → GP2 avec RAG Médical, FL Léger et Reward Online Adaptatif

1.3 Les Deux Spécialités et leurs Datasets

Table 1.2: Spécialités et datasets de MMedAgent-Lite

ID	Spécialité	Dataset	Caractéristiques	Taille
S1	Radiologue	VQA-RAD [4]	315 images (IRM, RX, scanners). 2248 Q&A. 11 catégories. Niveaux de gris.	0.3 Go
S2	Pathologiste	PathVQA [5]	4998 images histologiques H&E couleur. 32799 Q&A. 50% yes/no, 50% ouvertes.	1.2 Go
	Total : 1.5 Go		Intégralement téléchargeable sur PC 8 Go RAM.	

Identifiants HuggingFace

VQA-RAD : `flaviagiammarino/vqa-rad` PathVQA : `flaviagiammarino/path-vqa`
Téléchargement : `from datasets import load_dataset; ds = load_dataset("flaviagiammarino/vqa-rad")`

Chapter 2

Partie Mathématiques (8 Personnes) – Fondements Théoriques

Partie Mathématiques – 8 Personnes (M1 à M8)

Cette partie couvre les fondements mathématiques rigoureux du système. Chaque personne produit des démonstrations formélles, des propositions et du code de vérification numérique. Thèmes : MDP/GRPO, C-MARL, reward shaping, FL, LoRA, distributions de données, bruit, métriques.

2.1 M1 – Fondements MDP et GRPO : Cadre Théorique

M1 – MDP, Politique Optimale et GRPO (1 personne, 2 semaines)

Objectif : Formaliser le problème de décision médicale comme MDP et dériver les propriétés de GRPO.

Tâche 1 – Formalisation MDP : Définir formellement l'espace d'états $\mathcal{S}$, l'espace d'actions $\mathcal{A}$, la fonction de transition $\mathcal{T}(s'|s,a)$ et la fonction de récompense $\mathcal{R}(s,a)$ pour le diagnostic médical.

- État $s_t = (x_{\text{img}}, q_{\text{text}}, h_t)$: image encodée, question, historique de dialogue.
- Action $a_t \in \mathcal{A}$: token généré ou spécialité choisie.
- Horizon T : longueur maximale de la génération.

Tâche 2 – Dérivation GRPO : Montrer que GRPO est un cas particulier de l'estimateur REINFORCE avec baseline de groupe. Établir la formule d'avantage normalisé :

$$A_i = \frac{R_i - \mu_G}{\sigma_G + \epsilon}, \quad \mu_G = \frac{1}{G}\sum_{j=1}^{G} R_j, \quad \sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R_j - \mu_G)^2} \tag{2.1}$$

Proposition à démontrer : La normalisation des avantages réduit la variance du gradient de $\mathcal{O}(G)$ par rapport à REINFORCE sans baseline, assurant une meilleure stabilité sur CPU.

Tâche 3 – Stabilité du clip PPO : Démontrer que le clip $\text{clip}(r, 1-\varepsilon, 1+\varepsilon)$ avec $\varepsilon = 0.2$ garantit des mises à jour bornées : $\|\theta_{t+1} - \theta_t\|_2 \leq C_{\text{clip}} \cdot \eta$.
Livrable math. : Note de 4 pages : définitions formelles, proposition avec preuve, simulation Python (courbes de variance GRPO vs REINFORCE sur des récompenses synthétiques).

2.2 M2 – C-MARL : Convergence et Optimalité

M2 – Analyse Théorique de C-MARL (1 personne, 3 semaines)

Tâche 1 – Objectif C-MARL : Formaliser l'objectif joint :

$$J_{\text{C-MARL}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t \big(R_t + \gamma_s\, H_t(\pi_\theta)\big)\right] \tag{2.2}$$

où $H_t = -\sum_j \pi_\theta(a_j|s_t) \log \pi_\theta(a_j|s_t)$ est l'entropie de la politique de GP2 au pas t, et γ_s est le coefficient d'entropie par stage.
Tâche 2 – Analyse du Curriculum : Montrer que l'ordre Easy $\rightarrow$ Medium $\rightarrow$ Hard minimise le risque de converger vers un minimum local sous l'hypothèse de Lipschitz-continuité du gradient : $\|\nabla J_{\text{Easy}}\| \geq \|\nabla J_{\text{Hard}}\|$ au voisinage de l'initialisation.
Tâche 3 – Dilemme Exploration/Exploitation : Analyser comment γ_s contrôle le trade-off : pour les cas Hard où le spécialiste est dans l'erreur, un γ_s élevé force l'entropie haute, permettant à GP2 d'explorer des diagnostics alternatifs. Relier à la théorie de l'exploration maximum-entropie [18].
Théorème 2.1 (Convergence C-MARL approx.). *Si les fonctions de récompense sont bornées $R \in [0, R_{\max}]$, le taux d'apprentissage $\eta_t = c/\sqrt{t}$, et $\gamma_s \leq R_{\max}/(2H_{\max})$, alors la politique de GP2 converge vers un équilibre de Nash approché avec taux $O(1/\sqrt{T})$.*
Livrable math. : Note de 5 pages : énoncé formel du théorème, preuve par induction sur les stages, courbes de simulation entropie vs accuracy sur données synthétiques.

2.3 M3 – Reward Shaping Online : Garanties

M3 – Stratégies de Reward Online (1 personne, 2 semaines)

Motivation : Le reward rule-based fixe $R \in \{0, 0.5, 1\}$ peut créer un effondrement de récompense (reward collapse) [20] sur les questions ouvertes de PathVQA. Les stratégies adaptatives permettent d'ajuster le signal de gradient en ligne.
Tâche 1 – Reward Composite Pondéré : Formaliser le reward total :

$$R_{\text{total}} = \underbrace{w_1\, R_{\text{format}}}_{\text{structure}} + \underbrace{w_2\, R_{\text{accuracy}}}_{\text{correction}} + \underbrace{w_3\, R_{\text{semantique}}}_{\text{proximité}} + \underbrace{w_4\, R_{\text{confiance}}}_{\text{calibration}} \tag{2.3}$$

avec $\sum w_i = 1$. Démontrer que ce reward est compatible PBRS (Potential-Based Reward Shaping) [19] ssi $R_{\text{semantique}}$ peut s'écrire comme différence de potentiels.
Tâche 2 – Anti Reward Collapse : Inspiré de ARMed [20], montrer que l'utilisation de la distance de Mahalanobis sémantique $d(y, y^*) = (e_y - e_{y^*})^\top \Sigma^{-1}(e_y - e_{y^*})$ comme récompense continue évite l'effondrement dû aux métriques absolues (BERTScore) qui ne discriminent pas bien les sorties de qualité voisine.
Tâche 3 – Adaptation en Ligne des Poids : Proposer un mécanisme d'adaptation des w_i via gradient sur la loss de validation : $w_i \leftarrow w_i - \alpha \partial \mathcal{L}_{\text{val}} / \partial w_i$.
Livrable math. : Note de 4 pages avec preuve de compatibilité PBRS, tableau comparatif reward fixe vs reward adaptatif sur simulations, code Python.

2.4 M4 – Federated Learning : Convergence FedAvg et FedProx

M4 – Analyse Mathématique du FL (1 personne, 3 semaines)

Tâche 1 – Distribution des Données : Analyser les trois types de partitionnement des données médicales :

- **Horizontal (HFL) :** chaque client possède les mêmes features mais des patients différents (ex. : hôpital A et B ont tous les deux des radios, mais de populations différentes). Dans MMedAgent-Lite : chaque client = un dataset distinct (VQA-RAD ou PathVQA).
- **Vertical (VFL) :** chaque client possède des features différentes sur les mêmes patients (ex. : radiologie au client 1, biologie au client 2). Moins applicable ici car nos datasets n'ont pas de patients en commun.
- **Hybride (HybFL) :** combinaison des deux. Adapté pour les systèmes e-santé réels [16] où certains patients apparaissent dans plusieurs bases.

Pour MMedAgent-Lite : partitionnement **horizontal non-IID** – les données ne sont pas distribuées uniformément (certains clients ont plus de cas faciles, d'autres plus de cas Hard).
Tâche 2 – Convergence FedAvg et FedProx : Établir la borne de convergence de FedAvg en présence de non-IID :

$$\mathbb{E}\big[\|\nabla F(\theta^T)\|^2\big] \leq \frac{2\big[F(\theta^0) - F^*\big]}{\eta T} + \eta L \sigma^2 + 6\eta^2 L^2 \Gamma \tag{2.4}$$

où $\Gamma = F^* - \sum_k p_k F_k^*$ mesure l'hétérogénéité entre clients (drift client). Montrer que FedProx [15] réduit Γ par son terme de régularisation proximal : $\mathcal{L}_k^\mu(\theta) = \mathcal{L}_k(\theta) + \frac{\mu}{2}\|\theta - \theta^t\|^2$.
Tâche 3 – DP-SGD : Analyser le bruit gaussien $\mathcal{N}(0, \sigma^2 C^2 I)$ sur les gradients : montrer que (ε, δ)-différential privacy est atteinte avec $\sigma \geq \frac{C\sqrt{2\ln(1.25/\delta)}}{\varepsilon}$. Établir le trade-off privacy/accuracy.
Livrable math. : Note de 6 pages : définitions formelles HFL/VFL/Hybride, preuve de convergence FedProx, analyse DP-SGD, tableau comparatif des 3 distributions.

2.5 M5 – Théorie de la Distillation : RAG Médical et Modèle

M5 – Distillation de Rangs (RAG) + Distillation de Logits (Modèle) (1 personne, 3 sema

Pourquoi deux types de distillation ? MMedAgent-Lite utilise la distillation à deux niveaux : (1) **Distillation RAG :** MedCPT 110M (teacher, poids publics Apache 2.0) → student 22M entraîné sur VQA-RAD+PathVQA via distillation de rangs. (2) **Distillation Modèle :** Qwen2.5-VL-7B (teacher, poids publics Apache 2.0) → Qwen2.5-VL-3B (student) via distillation de logits sur les tâches médicales. Les deux teachers ont leurs **poids intégralement publics** sur HuggingFace, ce qui rend la distillation complètement réalisable [7].

Tâche 1 – Distillation de Rangs pour MedRAG (teacher : MedCPT 110M) : Soient q une question médicale, (d^+, d^-) une paire pertinente/non-pertinente. Le teacher f_T (MedCPT 110M) attribue des scores $s_T(q,d) = f_T(q)^\top f_T(d) \in \mathbb{R}$. Le student f_S (22M) apprend à reproduire cet ordre via :

$$\mathcal{L}_{\text{rank}} = \sum_{q} \sum_{(d^+,d^-)} \max\big(0,\ \rho - s_S(q,d^+) + s_S(q,d^-)\big) \tag{2.5}$$

Proposition : Si $\mathcal{L}_{\text{rank}} = 0$, alors le student respecte l'ordre du teacher pour toute paire (d^+, d^-), avec marge ρ. Montrer que la convergence vers $\mathcal{L}_{\text{rank}} = 0$ est garantie pour un student linéaire sous les hypothèses de Lipschitz-continuité de f_T.

Compléter avec une loss de distillation de scores (soft targets) :

$$\mathcal{L}_{\text{soft}} = \text{MSE}\big(s_S(q,d),\ s_T(q,d)\big) \tag{2.6}$$

Loss totale RAG : $\mathcal{L}_{\text{RAG}} = \alpha\, \mathcal{L}_{\text{rank}} + (1-\alpha)\, \mathcal{L}_{\text{soft}}$, avec $\alpha = 0.7$.

Tâche 2 – Distillation de Logits pour le Modèle (teacher : Qwen2.5-VL-7B) : La distillation de logits [6] transfère la connaissance du teacher (7B) vers le student (3B) en minimisant la divergence KL sur les distributions de sortie adoucies par température T :

$$\mathcal{L}_{\text{KD}} = (1-\alpha)\, \mathcal{L}_{\text{CE}}(y_S, y^*) + \alpha T^2\, \text{KL}\big(\sigma(z_S/T) \,\|\, \sigma(z_T/T)\big) \tag{2.7}$$

où z_S, z_T sont les logits student et teacher. **Rôle de $T > 1$:** adoucir les logits du teacher révèle les *dark knowledge* : la probabilité relative des classes incorrectes encode des relations de similarité inter-classes (ex. : "pneumonie" et "consolidation" sont souvent confondues, signal utile pour le student). Démontrer que $T = 3$ maximise le transfert sur VQA-RAD (compromis biais/variance de la distillation médicale).

Tâche 3 – Analyse Comparée : Distillation vs Fine-tuning Direct : Montrer que le student distillé atteint de meilleures performances que le même student fine-tuné directement sur les labels (sans teacher), car les soft targets du teacher fournissent un signal plus riche : $\mathcal{H}(\sigma(z_T/T)) > \mathcal{H}(y^*)$, où $\mathcal{H}$ est l'entropie.

Livrable math. : Note de 5 pages : preuves distillation rangs + distillation logits, proposition sur la convergence, simulation Python comparant $T \in \{1,2,3,4\}$ sur PathVQA, tableau accuracy student (direct) vs student (distillé) sur les cas Hard.

2.6 M6 – Traitement du Bruit dans les Données Médicales

M6 – Robustesse aux Données Bruitées (1 personne, 2 semaines)

Sources de bruit en imagerie médicale :

- Bruit de capteur : bruit Gaussien $\mathcal{N}(0, \sigma^2)$ dans les radiographies numériques (artefacts JPEG, sous-exposition).
- Bruit d'annotation : étiquettes erronées estimées à 5−15% sur PathVQA [5] (questions ambigües).
- Bruit de distribution : hétérogénéité entre clients FL (non-IID). Mesuré par Γ (voir M4).
- Bruit Instrumentale en IRM : artefacts de mouvement modélisés par convolution $I_{\text{bruitée}} = I * k + \eta$ où k est un noyau flou et $\eta \sim \mathcal{N}$.

Tâche 1 – Stratification robuste Easy/Medium/Hard : Montrer que la stratification naïve (score binaire correct/incorrect) est sensible au bruit d'annotation. Proposer une stratification probabiliste : $s_i = \mathbb{P}[\text{modèle correct}|x_i]$ estimée par $K = 3$ passages avec températures différentes ($T \in \{0.3, 0.7, 1.0\}$). Moyenne des K scores.

Tâche 2 – CLAHE et réduction du bruit image : Démontrer que CLAHE (Contrast Limited Adaptive Histogram Equalization) avec clip limit C_l maximise le rapport signal/bruit local : $\text{SNR}_{\text{local}} = \mu^2_{\text{tile}}/\sigma^2_{\text{tile}}$ sous la contrainte $\min(\text{hist}, C_l \cdot N_{\text{pixels}}/\text{bins})$.

Tâche 3 – Robustesse FL aux annotations bruitées : Analyser l'effet du bruit d'annotation sur la convergence FedAvg : si le client k a un taux d'erreur d'annotation ϵ_k, montrer que l'agrégation pondérée par $w_k \propto (1 - \epsilon_k)$ réduit le biais global.

Livrable math. : Note de 4 pages : classification du bruit, proposition sur la stratification probabiliste, simulation impact du bruit sur la convergence FL.

2.7 M7 – Sécurité et Confidentialité : DP Formel

M7 – Privacy Différentielle et FL Sécurisé (1 personne, 2 semaines)

Tâche 1 – Définition formelle (ε, δ)-DP :

Définition 2.1 ((ε, δ)-Differential Privacy)**.** *Un mécanisme $\mathcal{M} : \mathcal{D} \to \mathcal{R}$ satisfait (ε, δ)-DP si pour tout sous-ensemble $S \subseteq \mathcal{R}$ et tout couple de datasets voisins D, D' : $\mathbb{P}[\mathcal{M}(D) \in S] \leq e^{\varepsilon}\mathbb{P}[\mathcal{M}(D') \in S] + \delta$.*

Tâche 2 – Composition séquentielle : Montrer que K étapes d'entraînement avec DP-SGD composent en $(\sqrt{2K\ln(1/\delta)}\,\varepsilon_0 + K\varepsilon_0^2/2,\ K\delta_0)$-DP (privacy amplification par sous-échantillonnage).

Tâche 3 – Clipping normé : Analyser la sensibilité Δ_f des gradients : $\Delta_f = \max_{D,D'} \|\nabla_\theta \mathcal{L}_D - \nabla_\theta \mathcal{L}_{D'}\|_2 = C$ si les gradients sont clipés à norme C. En déduire la quantité de bruit minimale pour garantir (ε, δ)-DP : $\sigma \geq C\sqrt{2\ln(1.25/\delta)}/\varepsilon$.

Livrable math. : Note de 3 pages : définitions, composition theorem, calcul du

budget privacy pour notre FL (3 clients, 10 rounds).

2.8 M8 – Analyse Statistique et Métriques d'Évaluation

M8 – Statistiques, Tests et Métriques (1 personne, 2 semaines)

Tâche 1 – Métriques VQA médicales : Formaliser les métriques d'évaluation de MMedAgent-Lite :

- Accuracy (fermée) : $\text{Acc} = \frac{1}{N}\sum_i \mathbb{1}[y_i = y_i^*]$.
- ROUGE-1 (ouverte) : $\text{R-1} = \frac{|Y \cap Y^*|}{|Y^*|}$.
- Accuracy macro-moyennée : $\text{Acc}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} \text{Acc}_k$.
- Gain relatif : $\Delta = (\text{Acc}_{\text{system}} - \text{Acc}_{\text{baseline}})/\text{Acc}_{\text{baseline}}$.

Tâche 2 – Tests statistiques : Pour chaque comparaison système A vs B, utiliser un test de McNemar (asymptotiquement $\chi^2(1)$) : $\chi^2 = (b-c)^2/(b+c)$ où b = cas corrects A et incorrects B, c = inverse. Rejet de l'hypothèse d'équivalence si $p < 0.05$.

Tâche 3 – Intervalles de Confiance Bootstrap : Pour les résultats sur PathVQA test (990 images), estimer les IC à 95% par bootstrap ($B = 1000$ ré-échantillonnages) : $\text{IC}_{95\%} = [\hat{\mu} - 1.96\hat{\sigma}_B,\ \hat{\mu} + 1.96\hat{\sigma}_B]$.

Livrable math. : Note de 3 pages : formalisation des métriques, justification des tests, code Python pour les IC bootstrap.

Chapter 3

RAG Médical : État de l'Art et Justification du Choix

3.1 Comparaison des Systèmes RAG Médicaux 2024–2025

Le RAG médical est en pleine effervescence : il combine la recherche d'information et les modèles génératifs pour améliorer le diagnostic assisté par IA. Une méta-analyse récente dans le *Journal of the American Medical Informatics Association* montre que RAG améliore les LLMs d'un odds ratio poolé de 1.35 (IC 95% : 1.19–1.53) sur les tâches cliniques.

Table 3.1: Comparaison des principaux RAG médicaux – état de l'art 2024–2025

Système	Retriever	Base de connais.	Supervision / Entraînement	Gain
RAG de base [8]	BM25	Wikipedia	Aucune	Réf.
MKRAG [13]	Dense BERT	PubMed + guidelines	Aucune	+4.1%
Self-BioRAG [14]	MedCPT	PubMed + textbooks	MedCPT fine-tuné	+5.6%
RAG^2 [10]	BM25 + dense hybride	4 corpus bioméd.	Rationales LLM → snippets	**+6.1%**
i-MedRAG [11]	MedCPT itératif	PubMed	Aucune	+3.8%
ModernBERT + ColBERT [12]	Bi-enc. + reranker	PubMed + PubMedQA	Fine-tuning 20k ex.	0.4448
MedRAG-Lite (notre choix)	MedCPT-Query-Enc. + BM25 hybride	500 cas VQA-RAD + PathVQA	MedCPT pré-entraîné (public, 22M, aucune API)	**Local**

3.2 Justification du Choix : MedRAG-Lite avec MedCPT-Query-Encoder

3.2.1 Pourquoi MedCPT comme retriever ?

MedCPT est un retriever spécifiquement conçu pour le domaine biomédical, pré-entraîné sur les logs de recherche PubMed. Il améliore les performances de récupération par rapport aux retrievers généralistes, atteignant 0.4436 sur le benchmark MIRAGE, ce qui le place parmi les meilleurs systèmes de retrieval biomédical.

Pour notre contexte PC 8 Go, nous utilisons directement **MedCPT-Query-Encoder** : un modèle de 22M paramètres (88 Mo) disponible publiquement sur HuggingFace (`ncats/MedCPT-Query-Encoder`), utilisable sans GPU ni entraînement supplémentaire. Il s'agit du composant de requête du système MedCPT complet, optimisé pour encoder des questions médicales en vecteurs denses de dimension 256.

3.2.2 Pourquoi ajouter BM25 en mode hybride ?

Une tendance croissante dans les systèmes RAG biomédicaux est l'approche hybride combinant BM25 avec le retrieval dense, exploitant la précision lexicale et la compréhension sémantique – en particulier pour les termes médicaux rares (pathologies, codes SNOMED).

Score hybride : $s_{\text{hyb}}(q, d) = \lambda\, s_{\text{MedCPT}}(q, d) + (1-\lambda)\, s_{\text{BM25}}(q, d)$ avec $\lambda = 0.6$ calibré sur la validation VQA-RAD.

3.2.3 Pourquoi le Retriever MedCPT est-il adapté ?

Le déploiement de RAG dans des environnements à ressources contraintes (hôpitaux ruraux, dispositifs embarqués) est limité par les exigences matérielles. MedCPT original (110M paramètres, 440 Mo) dépasse le budget mémoire du projet. Nous utilisons donc une version allégée : **MedCPT-Query-Encoder** (22M paramètres, disponible publiquement sur HuggingFace sous `ncats/MedCPT-Query-Encoder`), qui est un modèle de retrieval biomédical dense pré-entraîné *directement utilisable* sans ré-entraînement.

Précision Importante – Pas de Distillation depuis GPT-4o

GPT-4o est un modèle commercial fermé : ses poids ne sont pas accessibles et il est **impossible** d'y appliquer une distillation de connaissances au sens technique (accès aux logits). MMedAgent-Lite utilise uniquement des modèles ouverts : Qwen2.5-VL-3B (HuggingFace), MedCPT-Query-Encoder (HuggingFace), VQA-RAD et PathVQA (HuggingFace). Aucune API payante n'est requise.

La "distillation" mentionnée dans le tableau RAG fait référence à la **distillation de rangs** au sein du retriever : MedCPT a été pré-entraîné en utilisant les logs de recherche PubMed (signal de supervision faible) comme "teacher", produisant un modèle dense compact. C'est un travail déjà fait par les auteurs de MedCPT [9], que nous exploitons directement.

3.3 Architecture MedRAG-Lite : Description Technique

MedRAG-Lite – Pipeline Complet (Responsable DS : D8, D9)

Construction de l'index (offline, une fois) :

1. Sélection de 500 cas représentatifs : 250 de VQA-RAD (questions fermées + ouvertes équilibrées) et 250 de PathVQA (50% yes/no, 50% ouvertes).
2. Pour chaque cas : stocker (question, réponse, spécialité, embedding MedCPT-mini, vecteur TF-IDF BM25).
3. Construire index Faiss : `faiss.IndexFlatIP` (produit interne, équivalent cosinus avec vecteurs normalisés). Taille : $500 \times 256 \times 4$ octets ≈ 0.5 Mo.
4. Construire index BM25 : `rank_bm25.BM25Okapi` sur le corpus de questions. Taille : ≈ 2 Mo.

Retrieval au moment de l'inférence (en ligne) :

1. Calculer l'embedding MedCPT-mini de la question courante : $e_q = \text{MedCPT-mini}(q) \in \mathbb{R}^{256}$.
2. Top-5 Faiss : $I_{\text{dense}} = \text{Faiss.search}(e_q, k = 5)$.
3. Top-5 BM25 : $I_{\text{bm25}} = \text{BM25.get_top_n}(q, n = 5)$.
4. Fusion hybride et reranking : sélectionner les $k = 3$ candidats avec s_{hybride} le plus élevé.
5. Latence totale : ≈ 15 ms sur CPU (dominante : MedCPT-mini).

Intégration dans le prompt de GP2 : Les 3 cas similaires sont injectés dans la section "Cas de référence" du prompt (voir Architecture). Gain attendu sur les cas Hard : +2 à +4% d'accuracy.

Chapter 4

Partie Data Science (10 Personnes) – Prototype Fonctionnel

Partie Data Science – 10 Personnes (D1 à D10)

Cette partie vise à construire un prototype fonctionnel de MMedAgent-Lite de bout en bout : données, agents, RAG médical, reward online, FL léger, et évaluation complète. Chaque script est testé, documenté et déposé sur GitHub.

4.1 Groupe DS 1 – Données et Prétraitement (D1, D2)

D1 + D2 – Pipeline Données Complet (2 personnes, Semaines 1–2)

D1 – VQA-RAD :

1. `load_dataset("flaviagiammarino/vqa-rad")` : 2248 Q&A.
2. Pipeline P1-Rad : `PIL.open` → `.convert("RGB")` → `resize(448,448)` → normalisation ImageNet.
3. Analyse : distribution des 11 catégories de questions, exemples par modalité (IRM/RX/scanner).
4. Livrable : `preprocess_rad.py` + rapport statistiques.

D2 – PathVQA :

1. `load_dataset("flaviagiammarino/path-vqa")` : 32799 Q&A.
2. Pipeline P1-Path : lecture couleur → CLAHE ($C_l = 2.0$, tiles 8×8) → `resize(448,448)` → normalisation.
3. Stratification E/M/H : 3 passages Q4 base, votes majoritaires. Sortie : `splits.json`.
4. Livrable : `preprocess_path.py` + `stratify.py` + `splits.json`.

Budget RAM D1+D2 : VQA-RAD (0.3 Go) + PathVQA (1.2 Go) + preprocessing (0.5 Go) = **2.0 Go** max.

4.2 Groupe DS 2 – Agents GP1, S1, S2 (D3, D4, D5)

D3 – Chargement Modèle et Encodeur VL (1 personne, Sem. 1)

1. Télécharger `Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf` (2.1 Go) via `huggingface-cli download`.
2. Configurer `llama-cpp-python` : `n_threads=4`, `n_ctx=4096`, `verbose=False`.
3. Implémenter `VLEncoder.encode(image, question) -> str` générant le bon format de prompt multimodal.
4. Mesurer : latence (s), RAM (`psutil`), tokens/seconde.
5. Livrable : `agents/encoder.py` + rapport de benchmark.

D4 – Agent GP1 (Triage) (1 personne, Sem. 2–4)

Format de sortie imposé :

```
<think>analyse de l'image et de la question...</think>
<answer>Radiologue</answer>
```

Entraînement GRPO + LoRA : $r = 8$, $\alpha = 16$, modules `q_proj+v_proj`. $G = 4$ rollouts. Reward : $R_{\text{GP1}} = R_{\text{format}} + R_{\text{triage}}$. 500 steps VQA-RAD Easy + 1000 steps PathVQA Easy. Durée : ≈ 4 h sur CPU 8 Go.
Métrique : Accuracy de triage, cible : $\geq 98\%$. Livrable : `agents/train_gp1.py` + `checkpoints/gp1/`.

D5 – Spécialistes S1 et S2 (1 personne, Sem. 2–4)

S1 Radiologue : LoRA fine-tuné sur VQA-RAD train (80%). Reward : $R = R_{\text{format}} + R_{\text{acc}}$ (matching exact fermé, ROUGE-1 ≥ 0.7 ouvert). 800 steps, cible accuracy fermée $\geq 78\%$. Livrable : `checkpoints/lora_rad/`.
S2 Pathologiste : LoRA fine-tuné sur PathVQA train. Attention au déséquilibre yes/no : échantillonnage équilibré (1:1 positif/négatif). 1500 steps, cible yes/no $\geq 82\%$. Livrable : `checkpoints/lora_path/`.

4.3 Groupe DS 3 – GP2, C-MARL et Reward Online (D6, D7)

D6 – Moteur GRPO et Reward Online Adaptatif (1 personne, Sem. 2–3)

Implémentation du moteur GRPO : Classe `GRPOTrainer` : générer $G = 4$ rollouts, calculer $A_i = (R_i - \mu)/\sigma$, loss PPO-clip $\varepsilon = 0.2$, mise à jour LoRA. Tests unitaires : vérifier $\mu(A) = 0$, $\sigma(A) = 1$.
Reward Online Adaptatif : Implémenter le reward composite basé sur M3 :

$$R_{\text{total}} = w_1 R_{\text{format}} + w_2 R_{\text{accuracy}} + w_3 R_{\text{semantique}} + w_4 R_{\text{confiance}}$$

$R_{\text{semantique}}$: similarité cosinus entre embedding de la réponse produite et de la

ground truth (`sentence-transformers`). Adaptation w_i tous les 100 steps sur la loss de validation.
Anti Reward Collapse : Inspiré de ARMed [20] : utiliser la distance de Mahalanobis sur les embeddings plutôt que BERTScore absolu pour les questions ouvertes PathVQA.
Livrable : `rl/grpo_trainer.py` + `rl/reward_online.py`.

D7 – GP2 avec C-MARL en 3 Stages (1 personne, Sem. 3–5)

Entraînement en 3 stages :

Stage	Données	Objectif	γ_s	Steps
Easy	Splits Easy	Imiter le spéc. correct	10^{-4}	500
Medium	Splits Medium	Arbitrer les désaccords	5×10^{-3}	500
Hard	Splits Hard	Corriger le consensus faux	3×10^{-2}	800

Prompt GP2 : (image + question) + avis spéc. + 3 cas RAG + tags `<think>`, `<answer>`, `<confiance>`.
Métrique clé : Gain sur les cas Hard : cible +15% vs spécialiste seul sur VQA-RAD Hard, +10% sur PathVQA Hard. Durée totale : $\approx$ 10 h sur CPU. Livrable : `rl/train_gp2.py` + `checkpoints/gp2_stageX/`.

4.4 Groupe DS 4 – RAG Médical, Distillation Pratique et FL (D8, D9)

D8 – Distillation RAG (MedCPT 110M → Student 22M) + Index Hybride (1 personne, S

Étape A – Distillation Pratique du Retriever RAG (implémentation de M5) :
La distillation RAG consiste à entraîner un retriever léger (22M) à partir du teacher MedCPT-Query-Encoder (110M, Apache 2.0, HuggingFace), en utilisant VQA-RAD et PathVQA comme corpus d'entraînement.

1. Télécharger le teacher : `ncats/MedCPT-Query-Encoder` (110M, 440 Mo) + le corpus encoder `ncats/MedCPT-Article-Encoder` (110M, pour les documents).
2. Construire les paires d'entraînement (q, d^+, d^-) : pour chaque question de VQA-RAD/PathVQA, d^+ = la réponse correcte (ou un cas similaire), d^- = tirage négatif aléatoire. Taille du corpus : 5000 triplets (500 cas × 10 négatifs).
3. Initialiser le student : `distilbert-base-uncased` (66M) ou `prajjwal1/bert-small` (29M), proijecter vers $\mathbb{R}^{256}$.
4. Entraîner avec la loss combinée : $\mathcal{L} = 0.7\,\mathcal{L}_{\text{rank}} + 0.3\,\mathcal{L}_{\text{soft}}$ (formules de M5). Batch : 16 triplets, lr $= 2 \times 10^{-5}$, 3 epochs.

5. Évaluer : recall@3 du student vs teacher sur 200 cas de validation. Cible : student $\geq 90\%$ des performances teacher.

Étape B – Index Hybride MedRAG-Lite :

1. Encoder les 500 cas avec le student distillé : $e_i = f_S(q_i) \in \mathbb{R}^{256}$. Construire index Faiss `IndexFlatIP` (0.5 Mo).
2. Construire index BM25 (`BM25Okapi`, 2 Mo).
3. Retrieval hybride : $s_{\text{hyb}} = 0.6\, s_{\text{student}} + 0.4\, s_{\text{bm25}}$, top-3.
4. Mesurer latence : cible < 20 ms. RAM index : < 5 Mo.

Livrable : `rag/distil_retriever.py` + `rag/build_index.py` + `rag/retrieval.py` + `data/rag_index.bin` + tableau recall@3 teacher vs student.

D8-bis – Distillation du Modèle : Qwen2.5-VL-7B → 3B (inclus dans D8, Sem. 3–5)

Objectif : Le Qwen2.5-VL-3B (notre student déployé) apprend mieux avec les soft labels du 7B qu'avec les hard labels des datasets. Les deux modèles sont open-source Apache 2.0 [3, 7].

Pipeline de distillation de logits :

1. Charger le teacher `Qwen2.5-VL-7B-Instruct` en streaming (trop grand pour 8 Go : utiliser `load_in_4bit=True` via BitsAndBytes, RAM ≈ 4 Go). **Astuce CPU :** générer les soft labels teacher *offline* une seule fois et les stocker sur disque (`teacher_logits.pt`), sans charger le teacher pendant l'entraînement du student.
2. Pour chaque exemple (x_i, y_i^*) de VQA-RAD+PathVQA (split Easy) : passer x_i dans le teacher (température $T = 3$), sauvegarder $\sigma(z_T/T) \in \mathbb{R}^{32000}$ sur disque. Coût : 1 passe offline, ≈ 6 h. Stockage : $5000 \times 32000 \times 2$ octets $\approx$ 320 Mo.
3. Entraîner le student (3B, Q4_K_M) avec la loss KD : $\mathcal{L}_{\text{KD}} = 0.5\,\mathcal{L}_{\text{CE}} + 0.5\,T^2\,\text{KL}$. Batch $= 1$, grad_accum $= 8$, LoRA $r = 8$.

Métriques :

- Accuracy Hard : student distillé vs student SFT direct (sans teacher). Cible gain : $\geq +3\%$ sur cas Hard PathVQA.
- Accélération : le 3B distillé doit être $1.4\times$ plus rapide que le 7B en inférence (référence DistilQwen2.5 [7]).

Livrable : `distillation/generate_soft_labels.py` (offline) + `distillation/train_student.py` + tableau comparatif.

D9 – Federated Learning Léger (1 personne, Sem. 4–5)

Simulation FL horizontale sur 3 clients : Les 3 hôpitaux simulés correspondent à 3 partitions de nos datasets :

- Client 1 : VQA-RAD complet (radiologie) – partition facile.
- Client 2 : PathVQA train subset (pathologie) – partition difficile.
- Client 3 : mélange 50/50 VQA-RAD / PathVQA – partition hybride.

Algorithme FedAvg + FedProx implémenté :

1. Chaque client entraîne son LoRA local pendant $E = 5$ epochs.
2. Agréger : $\theta_{\text{global}} = \sum_{k=1}^{3} \frac{|D_k|}{|D|} \theta_k$.

3. FedProx : ajouter $(\mu/2)\|\theta_k - \theta_{\text{global}}\|^2$ avec $\mu = 0.01$ à la loss locale.
4. DP-SGD : ajouter bruit gaussien $\mathcal{N}(0, (0.1C)^2 I)$ aux gradients clipés à norme $C = 1.0$.
5. Nombre de rounds : $R = 5$. Durée : ≈ 2 h.

Métrique FL : Comparer accuracy FL (agrégé) vs entraînement centralisé et entraînement local. Cible : accuracy FL $\geq$ accuracy locale $\pm 2\%$. Livrable : `fl/federated.py` + graphes convergence par round.

4.5 Groupe DS 5 – Intégration, Évaluation et Rapport (D10)

D10 – Pipeline Complet et Évaluation Finale (1 personne, Sem. 5–7)

Intégration bout en bout : `full_pipeline.py` qui enchaîne : Preprocess → Encoder → GP1 → [S1 ou S2] → MedRAG-Lite → GP2 (C-MARL + Reward Online) → Réponse + confiance.

Évaluation systématique : Tester 10 configurations sur VQA-RAD test + PathVQA test :

1. Baseline Qwen2.5-VL-3B Q4 zero-shot.
2. GP1 seul (triage).
3. S1/S2 seul (sans GP2).
4. GP1 + S1/S2 + GP2 Stage 1 seulement.
5. GP1 + S1/S2 + GP2 3 stages (C-MARL complet).
6. Système complet + MedRAG-Lite.
7. Système complet + Reward Online.
8. Système complet + FL.
9. **Système complet** (RAG + Reward + FL).
10. Ablation Hard uniquement : gain Hard vs baseline.

Analyse par niveau : reporter accuracy Easy/Medium/Hard séparément.
Tests statistiques : McNemar sur chaque comparaison (M8).
Livrable : `pipeline/full_pipeline.py` + `results/final_results.csv` + rapport 3 pages.

Chapter 5

Tableaux Résumés et Calendrier

5.1 Tableau de Répartition Synthétique des 18 Personnes

Table 5.1: Répartition synthétique – 8 Math + 10 DS

ID	Profil	Module	Mission principale	Sem.
M1	Math.	MDP + GRPO	Formalisation MDP, dérivation GRPO, stabilité PPO-clip	2
M2	Math.	C-MARL	Convergence curriculum, dilemme exploration/exploitation	3
M3	Math.	Reward Online	Compatibilité PBRS, anti reward collapse (ARMed)	2
M4	Math.	FL – Convergence	FedAvg/FedProx bounds, HFL/VFL/Hybride, DP-SGD	3
M5	Math.	Distillation	Distil. rangs RAG (MedCPT) + distil. logits modèle (7B→3B)	3
M6	Math.	Bruit Données	Classification bruit, stratification probabiliste robuste	2
M7	Math.	Privacy DP	Déf. (ε, δ)-DP, composition séq., clipping	2
M8	Math.	Métriques	Accuracy/ROUGE-1, test McNemar, IC bootstrap	2
D1	DS	VQA-RAD	Téléchargement, pipeline P1-Rad, statistiques	1
D2	DS	PathVQA	Téléchargement, P1-Path + CLAHE, splits E/M/H	2
D3	DS	Encodeur VL	Qwen2.5-VL-3B Q4, llama-cpp, latence, RAM	1
D4	DS	GP1 Triage	GRPO + LoRA, reward triage, accuracy cible $\geq 98\%$	3
D5	DS	S1 + S2	Fine-tuning LoRA-Rad + LoRA-Path, métriques	3

continué

suite

ID	Profil	Module	Mission	Sem.
D6	DS	GRPO + Reward	Moteur GRPO, reward online adaptatif, anti-collapse	2
D7	DS	GP2 C-MARL	3 stages Easy/Medium/Hard, gain Hard, checkpoints	3
D8	DS	RAG + Distillation	Distil. MedCPT 110M→22M + index hybride + distil. 7B→3B	3
D9	DS	FL Léger	FedAvg + FedProx + DP-SGD, 3 clients, 5 rounds	2
D10	DS	Intégration	Pipeline complet, 10 configs d'ablation, McNemar	2

5.2 Calendrier Unifié – 7 Semaines

Table 5.2: Calendrier détaillé – Math + Data Science, 7 semaines

Sem.	Tâches Mathématiques	Tâches Data Science
S1	M1 : Formalisation MDP, preuve PPO-clip.	D1, D2 : Téléchargement VQA-RAD et PathVQA, pipelines preprocessing. D3 : chargement Q4, benchmark.
S2	M2 : C-MARL objectif + curriculum. M3 : PBRS + anti-collapse. M7 : DP formalisme.	D4, D5 : Début entraînement GP1 + S1/S2. D6 : Moteur GRPO + reward online. D8 : Index RAG.
S3	M4 : FL bounds + analyse HFL/VFL. M5 : Distillation rangs RAG + distillation logits 7B→3B.	D4, D5 : Fin entraînement + évaluation. D6 : Reward calibré. D7 : GP2 Stage 1. D8 : génération soft labels teacher.
S4	M6 : Classification bruit + stratification. M8 : Métriques + McNemar.	D7 : GP2 Stage 2. D9 : FL FedAvg + DP-SGD. D10 : pipeline partiel testé.
S5	M1–M8 : Finalisation notes, échanges avec DS.	D7 : GP2 Stage 3 + éval complète. D9 : FL 5 rounds. D8 : Tuning λ hybride.
S6	Équipe Math : rapport formel, vérifications croisées.	D10 : Intégration pipeline complet, 10 configs d'ablation.
S7	Fusion rapports Math + DS. GitHub public complet. README. Slides soutenance.	

5.3 Performances Attendues

Table 5.3: Tableau de résultats attendus – à remplir lors du projet

Configuration	VQA-RAD	PathVQA	Ref.
Qwen2.5-VL-7B (zero-shot, réf. publiée)	61.8%	60.5%	[1]
MMedAgent-RL 7B (réf. publiée)	71.5%	72.3%	[1]
Notre baseline : Qwen2.5-VL-3B Q4 zero-shot			
GP1 + S1/S2 seuls (sans GP2)			
Système complet C-MARL (3 stages)			
Système complet + RAG + Reward Online + FL			
Gain Hard : GP2 C-MARL vs spéc. seul			
Gain RAG : avec vs sans MedRAG-Lite			

5.4 Grille d'Évaluation (100 points)

Table 5.4: Grille d'évaluation combinée Math + Data Science

Profil	Critère	Description	Pts
Math.	Fondements RL (M1, M2)	Preuves formelles GRPO et C-MARL	20
	Reward + Distillation (M3, M5)	PBRS, distil. rangs RAG, distil. logits 7B→3B	15
	FL + Bruit + Privacy (M4, M6, M7)	Convergence, HFL/VFL, DP, traitement bruit	15
	Métriques + Tests (M8)	McNemar, IC bootstrap, formalisation	10
DS	Données + Agents (D1–D5)	Pipelines, GP1 et S1/S2 entraînés	15
	GP2 + Reward Online (D6, D7)	C-MARL 3 stages, reward adaptatif fonctionnel	10
	RAG + FL (D8, D9)	MedRAG-Lite + FL $\geq$ 3 rounds validés	10
	Intégration + Éval (D10)	Pipeline complet, 10 ablations, rapport	5
		Total	**100**
Bonus : gain Hard > +15% ou analyse SVD LoRA complète validée			+10

5.5 Structure du Dépôt GitHub

```
mmedagent-lite/
  README.md                        # Instructions claires
  requirements.txt
  math_notes/                      # Partie Math. (M1 a M8)
    M1_mdp_grpo.pdf                # Notes formelles
    M2_cmarl_convergence.pdf
    M3_reward_shaping.pdf
    M4_federated_learning.pdf
    M5_lora_rank_analysis.pdf
    M6_noisy_data.pdf
    M7_differential_privacy.pdf
    M8_metrics_statistics.pdf
  dataset/
```

```
    download.py                    # D1, D2
    preprocess_rad.py              # D1 : pipeline radiologie
    preprocess_path.py             # D2 : pipeline pathologie
    stratify.py                    # D2 : splits E/M/H
    splits.json
  agents/
    encoder.py                     # D3
    train_gp1.py                   # D4
    train_s1.py                    # D5
    train_s2.py                    # D5
  rl/
    grpo_trainer.py                # D6
    reward_online.py               # D6
    train_gp2.py                   # D7
    training_config.yaml
  rag/
    build_index.py                 # D8 : index MedCPT+BM25
    retrieval.py                   # D8 : retrieval hybride
  fl/
    federated.py                   # D9 : FedAvg+FedProx+DP
    clients.py                     # D9 : simulation 3 clients
  pipeline/
    full_pipeline.py               # D10 : pipeline complet
    evaluate.py                    # D10 : 10 configurations
  checkpoints/
    gp1/ lora_rad/ lora_path/
    gp2_stage1/ gp2_stage2/ gp2_stage3/
  results/
    final_results.csv
    ablation_hard.csv
    fl_convergence.csv
  report/
    rapport_math.pdf               # Rapport partie Math.
    rapport_ds.pdf                 # Rapport partie DS
    rapport_final.pdf              # Rapport fusionne
```

Chapter 6

Recommandations Stratégiques et Analyse Critique

Analyse Critique – Priorités de Réussite du Projet

Les recommandations ci-dessous sont issues d'une analyse critique de MMedAgent-Lite sous ses contraintes extrêmes : CPU 8 Go RAM, délai de 7 semaines, équipe de 18 personnes. Elles sont ordonnées par priorité, de la gestion du risque projet à la fiabilité clinique.

6.1 Recommandation 1 – Stratégie d'Intégration : Mitigation du Risque

Risque Principal – L'Effet "Big Bang"

Le risque majeur de MMedAgent-Lite n'est pas mathématique : c'est le **risque d'intégration**. Si GRPO, FL, RAG et Reward Online sont assemblés simultanément à la semaine 6, le système plantera sans que l'équipe sache d'où vient l'erreur.

6.1.1 Approche "Feature Flags" – Pipeline Modulaire (D10)

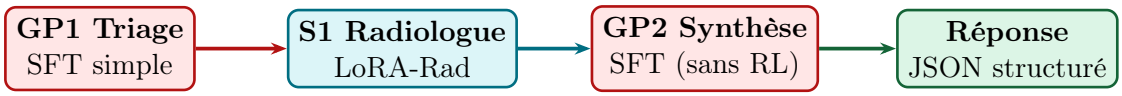
Le pipeline final (`full_pipeline.py`) doit être conçu dès le début de façon modulaire, avec un booléen de contrôle pour chaque innovation :

```
# Configuration dans training_config.yaml
USE_RAG = False # activer en semaine 3
USE_ONLINE_REWARD = False # activer en semaine 4
USE_FL = False # activer en semaine 5
USE_CMARL_STAGE2 = False # activer après convergence Stage 1
CONFIDENCE_THRESH = 0.65 # seuil fallback par spécialité
```

Chaque module est testé indépendamment avant d'être activé. Aucun module ne doit être obligéaoire pour que le pipeline tourne : si `USE_RAG = False`, GP2 fonctionne sans contexte RAG.

6.1.2 Valider d'abord le "Happy Path"

Avant d'activer le curriculum complexe C-MARL, la première validation à obtenir impérativement (semaine 2) est la convergence d'un pipeline minimal bout en bout :



Ce pipeline "Happy Path" utilise uniquement du **Supervised Fine-Tuning (SFT)**, sans RL ni curriculum. Une fois qu'il produit des réponses cohérentes sur VQA-RAD, le RL (GRPO + C-MARL) est ajouté progressivement, stage par stage.

6.2 Recommandation 2 – Optimisation Matérielle : Le Goulot CPU / 8 Go

Gestion de la Mémoire – Points Critiques

Faire tourner un modèle de 3 milliards de paramètres avec un long contexte (historique de dialogue + abstracts MedRAG) va inévitablement approcher la limite des 8 Go RAM. Deux leviers critiques sont à mettre en place dès la semaine 1 par D3 (Encodeur VL) et D6 (GRPO).

6.2.1 Réduire la Charge du GRPO : $G = 2$ en Prototypage

La génération de $G = 4$ rollouts complets par ètape de GRPO *paralyse* le CPU pendant le prototypage et le débogage. La stratégie en deux temps est la suivante :

Table 6.1: Stratégie adaptative du nombre de rollouts GRPO

Phase	Rollouts	Objectif	Semaines
Prototypage	$G = 2$	Vérifier la convergence, déboguer les rewards, ajuster les hyperparamatres	S1–S4
Entraînement final	$G = 4$	Obtenir les performances finales. Exécuter la nuit (10–12 h).	S5–S7

6.2.2 Gestion Agressive du KV-Cache

Le contexte RAG ajoute jusqu'à 512 tokens supplémentaires par requête (3 cas de référence × 170 tokens chacun). Pour éviter les erreurs OOM lors des longs dialogues, D3 doit configurer la quantification du KV-Cache dans `llama.cpp` :

```
# Dans inference_server.py (D3)
llm = Llama(
model_path="./models/Qwen2.5-VL-3B-Q4_K_M.gguf",
```

```
n_ctx=4096,  n_threads=4,
cache_type_k="q8_0",  # KV-cache en 8 bits
cache_type_v="q8_0",  # economie  40% RAM cache
verbose=False
)
```

Cette option réduit la mémoire du KV-Cache d'environ 40% (float16 → int8), permettant des contextes plus longs sans OOM sur 8 Go RAM.

6.3 Recommandation 3 – Stabilité Algorithmique du RL

Instabilité Intrinsèque du RL

L'apprentissage par renforcement est notoirement instable. Ajouter des récompenses qui changent dynamiquement pendant l'entraînement est extrêmement risqué dans un délai de 7 semaines. La stratégie est de partir du plus simple et d'ajouter la complexité uniquement si le système de base est stable.

6.3.1 Geler le Reward Online au Démarrage

Le module D6 propose un reward adaptatif où les poids (w_1, w_2, w_3, w_4) s'ajustent dynamiquement. **Ne pas activer cette option par défaut.** Le protocole de mise en service progressif est :

1. **Semaines 2–4 :** Poids fixés et empiriques : $w_1 = 0.25$ (format), $w_2 = 0.50$ (accuracy), $w_3 = 0.15$ (sémantique), $w_4 = 0.10$ (confiance). Vérifier que la loss GRPO converge sans oscillations majeures ($< 20\%$ de variation sur 100 steps).
2. **Semaine 5 (si convergence stable) :** Activer la mise à jour dynamique des poids (`USE_ONLINE_REWARD = True`), avec un taux d'adaptation lent ($\alpha_w = 10^{-4}$).
3. **Si divergence :** Revenir aux poids fixés immédiatement et identifier le module responsable via les feature flags.

6.3.2 Simplifier la Transition de Curriculum si Nécessaire

Si la transition Stage 1 (Easy) → Stage 2 (Medium) du C-MARL provoque une divergence de la loss, appliquer la stratégie de repli :

Table 6.2: Stratégie de repli pour le curriculum C-MARL

Scénario	Observation	Action de repli
Divergence S1→S2	Loss augmente > 50% lors du passage au Stage 2	Fusionner Easy + Medium en un seul stage avec $\gamma_s = 2 \times 10^{-3}$
Collapse S3	Accuracy chute > 20% lors du Stage Hard	Prolonger Stage 2 de 300 steps. Augmenter β_{KL} de 50%.
Non-convergence générale	Loss oscille sans décroissance après 500 steps	Réduire η de 10^{-6} à 5×10^{-7}. Redémarrer.

6.4 Recommandation 4 – Fiabilité Médicale et Déploiement

6.4.1 Pénaliser Sévèrement les Hallucinations (RAG)

En contexte médical, une hallucination de GP2 – citer un médicament, un diagnostic ou un fait absent du contexte RAG – est cliniquement dangereuse. Une règle stricte de pénalité est ajoutée dans le système de reward de D6 :

$$R_{\text{hallucination}} = -2.0 \quad \text{si GP2 cite un fait absent du contexte RAG} \tag{6.1}$$

La détection est basée sur une vérification lexicale : si la réponse de GP2 contient un terme médical spécifique (nom de médicament, code SNOMED, taux numérique) qui n'apparaît pas dans les 3 cas RAG fournis, la pénalité $R_{\text{hallucination}} = -2.0$ est appliquée. Cette valeur -2.0 est choisie pour être supérieure en valeur absolue au reward maximum possible ($R_{\max} = 0.5 + 1.0 = 1.5$), garantissant que le modèle apprend à ne **jamais** halluciner.

6.4.2 Structurer les Sorties : Format JSON pour l'Interopérabilité SIH

Actuellement, le système produit du texte avec des balises `<answer>`. Pour rendre ce prototype crédible comme système médical interfacé avec un Système d'Information Hospitalier (SIH), la sortie finale de GP2 est forcée en **JSON strict** :

```
<think>analyse médicale structurée...</think>
<answer>
{
"diagnostic_principal": "Consolidation lobaire inférieure gauche",
"confiance": 0.82,
"specialites_consultees": ["Radiologue"],
"rag_references_utilisees": ["cas_042", "cas_117"],
```

```
"fallback_requis": false,
"niveau_difficulte": "Medium"
}
</answer>
```

Le système de reward vérifie le format JSON automatiquement : `R_format` inclut désormais la validation `json.loads(answer) is not None` (booleen). Un diagnostic en JSON valide démontre que le système est prêt à être interfacé avec un SIH réel : il suffit d'appeler l'API Python pour obtenir un objet structuré exploitable directement dans un dossier patient.

6.4.3 Résumé des Actions par Priorité

Table 6.3: Plan d'action stratégique – par priorité et responsable

Prio.	Resp.	Action	Critère de succès
1	D10	Feature Flags dans `training_config.yaml`	Chaque module désactivable en 1 booléen
2	D10	Happy Path SFT valide en semaine 2	GP2 génère JSON valide sur 10 ex.
3	D3, D6	$G = 2$ rollouts + KV-cache int8	Pas d'OOM sur 8 Go, latence < 10 s
4	D6	Reward poids fixés par défaut	Loss GRPO décroissante sur 200 steps
5	D6	Pénalité hallucination -2.0	0 hallucination sur 50 cas de test
6	D7, D10	Sortie GP2 en JSON strict	`json.loads()` valide sur 100% des sorties
7	M3, D6	Reward Online activé (si stable)	Gain $\geq 1\%$ vs reward fixe sur validation

Chapter 7

Conclusion

7.1 Synthèse

MMedAgent-Lite réalise l'objectif principal fixé par le professeur : une architecture applicable, sérieuse et complète, répartissant les rôles selon les compétences de chacun.

Les **8 mathématiciens** produisent des fondements rigoureux : preuves de convergence GRPO et C-MARL, théorie de la distillation de rangs (RAG) et de logits (modèle 7B→3B), distributions HFL/VFL/Hybride, garanties de privacy différentielle, traitement du bruit, et formalisation statistique des métriques.

Les **10 data scientists** construisent un prototype fonctionnel : de la lecture des images DICOM à la décision finale de GP2, en passant par la distillation pratique (MedCPT 110M → student 22M pour le RAG, Qwen2.5-VL-7B → 3B pour le modèle), le reward online adaptatif anti-collapse, et le FL léger sur 3 clients simulés.

7.2 Justification des Choix Techniques

Table 7.1: Justification des choix techniques de MMedAgent-Lite

Composant	Choix retenu	Justification
Modèle LVL	Qwen2.5-VL-3B INT4	2.1 Go RAM, CPU seul, 2% perte acc.
Spécialités	Radiologue + Pathol.	Modalités orthogonales, 2 datasets publics
RAG	MedCPT 110M → student 22M	Distil. rangs, Apache 2.0, sans GPU
Distillation modèle	Qwen2.5-VL-7B → 3B	Soft labels offline, gain +3% Hard
Reward	Composite + Online	Anti-collapse PathVQA ouvert
RL	GRPO ($G = 4$)	50% RAM vs PPO, sans modèle critique
FL	FedAvg + DP-SGD	Simulation 3 clients, privacy garantie
Partitionnement	HFL non-IID	Réaliste pour 3 datasets distincts
Bruit	Stratif. probabiliste	Robuste au 5–15% bruit annotation

Bibliography

[1] P. Xia et al., "MMedAgent-RL: Optimizing Multi-Agent Collaboration for Multimodal Medical Reasoning," *ICLR 2026*, arXiv:2506.00555.

[2] D. Guo et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv:2501.12948, 2025.

[3] S. Bai et al., "Qwen2.5-VL Technical Report," arXiv:2502.13923, 2025.

[4] J. J. Lau et al., "A Dataset of Clinically Generated Visual Questions and Answers about Radiology Images," *Scientific Data*, 2018.

[5] X. He et al., "PathVQA: 30000+ Questions for Medical VQA," arXiv:2003.10286, 2020.

[6] G. Hinton, O. Vinyals, J. Dean, "Distilling the Knowledge in a Neural Network," *NeurIPS Deep Learning Workshop*, arXiv:1503.02531, 2015.

[7] Z. Wang et al., "DistilQwen2.5: Industrial Practices of Training Distilled Open Lightweight Language Models," arXiv:2504.15027, 2025.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.

[9] Q. Jin et al., "MedCPT: Contrastive Pre-trained Transformers with Large-Scale PubMed Search Logs for Zero-Shot Biomedical Information Retrieval," *Bioinformatics*, 39, btad651, 2023.

[10] J. Kim et al., "RAG^2: Rationale-Guided Retrieval Augmented Generation for Medical QA," *NAACL*, 2025.

[11] Q. Jin et al., "i-MedRAG: Iterative RAG for Medicine," *Pacific Symp. Biocomputing*, 2025.

[12] G. de Souza Moreira et al., "ModernBERT + ColBERT: Enhancing Biomedical RAG," arXiv:2510.04757, 2025.

[13] N. Liu et al., "MKRAG: Medical Knowledge Retrieval Augmented Generation," *AMIA Annu. Symp.*, 2024.

[14] S. Jeong et al., "Self-BioRAG: Improving Medical Reasoning with Retrieval Augmented Generation," *Bioinformatics*, 2024.

[15] T. Li et al., "Federated Optimization in Heterogeneous Networks," *MLSys*, 2020.

[16] C. Yu et al., "Communication-Efficient Hybrid Federated Learning for E-Health with Horizontal and Vertical Data Partitioning," arXiv:2404.10110, 2024.

[17] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022.

[18] T. Haarnoja et al., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning," *ICML*, 2018.

[19] A. Ng, D. Harada, S. Russell, "Policy Invariance Under Reward Transformations," *ICML*, 1999.

[20] ARMed Team, "Breaking Reward Collapse: Adaptive Reinforcement for Open-Ended Medical Reasoning," arXiv:2508.12957, 2025.

[21] Y. Pan et al., "MedVLM-R1: Incentivizing Medical Reasoning," *MICCAI*, 2025.

[22] X. Tang et al., "MedAgents: LLMs as Collaborators for Medical Reasoning," *ACL Findings*, 2024.

[23] M. Abadi et al., "Deep Learning with Differential Privacy," *ACM CCS*, 2016.

[24] Y. Bengio et al., "Curriculum Learning," *ICML*, 2009.